

Probe: Liveness, Readiness e Startup

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Descrivere il ruolo e il comportamento di `livenessProbe`, `readinessProbe` e `startupProbe`
- Configurare i quattro tipi di handler: `exec`, `httpGet`, `tcpSocket`, `grpc`
- Scegliere i valori appropriati per `initialDelaySeconds`, `periodSeconds`, `timeoutSeconds`, `failureThreshold`
- Spiegare l'impatto delle probe sul rolling update e sulla disponibilità del servizio
- Identificare gli anti-pattern comuni (liveness troppo aggressiva, dipendenze esterne nella liveness)
- Configurare `startupProbe` per applicazioni Java/Spring con JVM warm-up su ARO

Teoria e Concetti Chiave

Perché le Probe

Kubernetes non sa se un container è "funzionante" guardando solo il processo in esecuzione. Un processo può essere attivo ma bloccato (deadlock), o in fase di bootstrap (non ancora pronto a ricevere traffico). Le probe risolvono questi scenari:

Probe	Trigger azione	Azione del kubelet	Use case principale
<code>livenessProbe</code>	Fallisce <code>failureThreshold</code> volte	Kill + restart del container	Rilevare deadlock, blocchi applicativi
<code>readinessProbe</code>	Fallisce	Rimuove il Pod dagli Endpoint del Service	App non pronta: db in attesa, cache warming
<code>startupProbe</code>	Ha successo → abilita liveness + readiness	(blocca le altre probe finché non passa)	App con avvio lento (JVM, migration DB)

Tipi di Handler

Handler	Descrizione	Use case
<code>exec</code>	Esegue un comando nel container; successo se exit code = 0	Script di health check custom, verifica file
<code>httpGet</code>	HTTP GET verso un endpoint; successo se status code 200-399	REST API health endpoint (<code>/actuator/health</code>)
<code>tcpSocket</code>	Tenta connessione TCP a una porta; successo se porta raggiungibile	Database, message broker (non HTTP)
<code>grpc</code>	Health check gRPC standard; successo se risposta SERVING	Servizi gRPC

Campi di Configurazione

Campo	Default	Descrizione
<code>initialDelaySeconds</code>	0	Attesa prima del primo check (deprecato con <code>startupProbe</code>)
<code>periodSeconds</code>	10	Frequenza dei check
<code>timeoutSeconds</code>	1	Timeout della singola probe
<code>failureThreshold</code>	3	Numero di fallimenti consecutivi prima dell'azione
<code>successThreshold</code>	1	Numero di successi per tornare "healthy" (solo readiness, >1 raro)

`livenessProbe` — Rileva Container Bloccati

Il kubelet riavvia il container se la probe fallisce `failureThreshold` volte. Utile per rilevare:

- Deadlock (processo attivo ma bloccato)
- Memory leak che porta l'app in stato non funzionale
- Stato corrotto non recuperabile senza riavvio

⚠ Attenzione (Anti-pattern): Non configurare la `livenessProbe` su un endpoint che fa chiamate a dipendenze esterne (database, API esterne). Se il database è down, la liveness farebbe riavviare tutti i Pod inutilmente, peggiorando la situazione (thundering herd).

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness    # ← endpoint separato da readiness
    port: 8080
    httpHeaders:
      - name: Accept
        value: application/json
  periodSeconds: 10
  timeoutSeconds: 3
  failureThreshold: 3
```

readinessProbe — Controlla la Disponibilità al Traffico

Il Pod viene rimosso dagli Endpoint del Service (non riceve traffico) se la probe fallisce. Utile per:

- App che impiega tempo a scaldarsi (cache warming, connection pool)
- App che deve attendere la disponibilità del database
- Applicazioni con circuito "aperto" temporaneo

💡 **Suggerimento:** `readinessProbe` può fallire e tornare healthy senza riavvio del Pod. È il meccanismo corretto per gestire overload temporanei.

```
readinessProbe:
  httpGet:
    path: /actuator/health/readiness # ← endpoint che verifica dipendenze
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
  failureThreshold: 3
  successThreshold: 1
```

startupProbe — Gestisce Avvii Lenti

Blocca la valutazione di `livenessProbe` e `readinessProbe` fino a quando non ha successo. Permette di impostare `initialDelaySeconds` bassi sulle altre probe senza rischiare riavvii prematuri durante il bootstrap:

Formula del tempo massimo di avvio:

$$T_{max} = failureThreshold \times periodSeconds$$

Esempio: `failureThreshold: 30`, `periodSeconds: 10` → max 300 secondi (5 minuti) per avviarsi.

```
startupProbe:
  httpGet:
    path: /actuator/health
    port: 8080
  failureThreshold: 30 # ← 30 × 10s = 5 min max per avviarsi
  periodSeconds: 10
```

Impatto sul Rolling Update

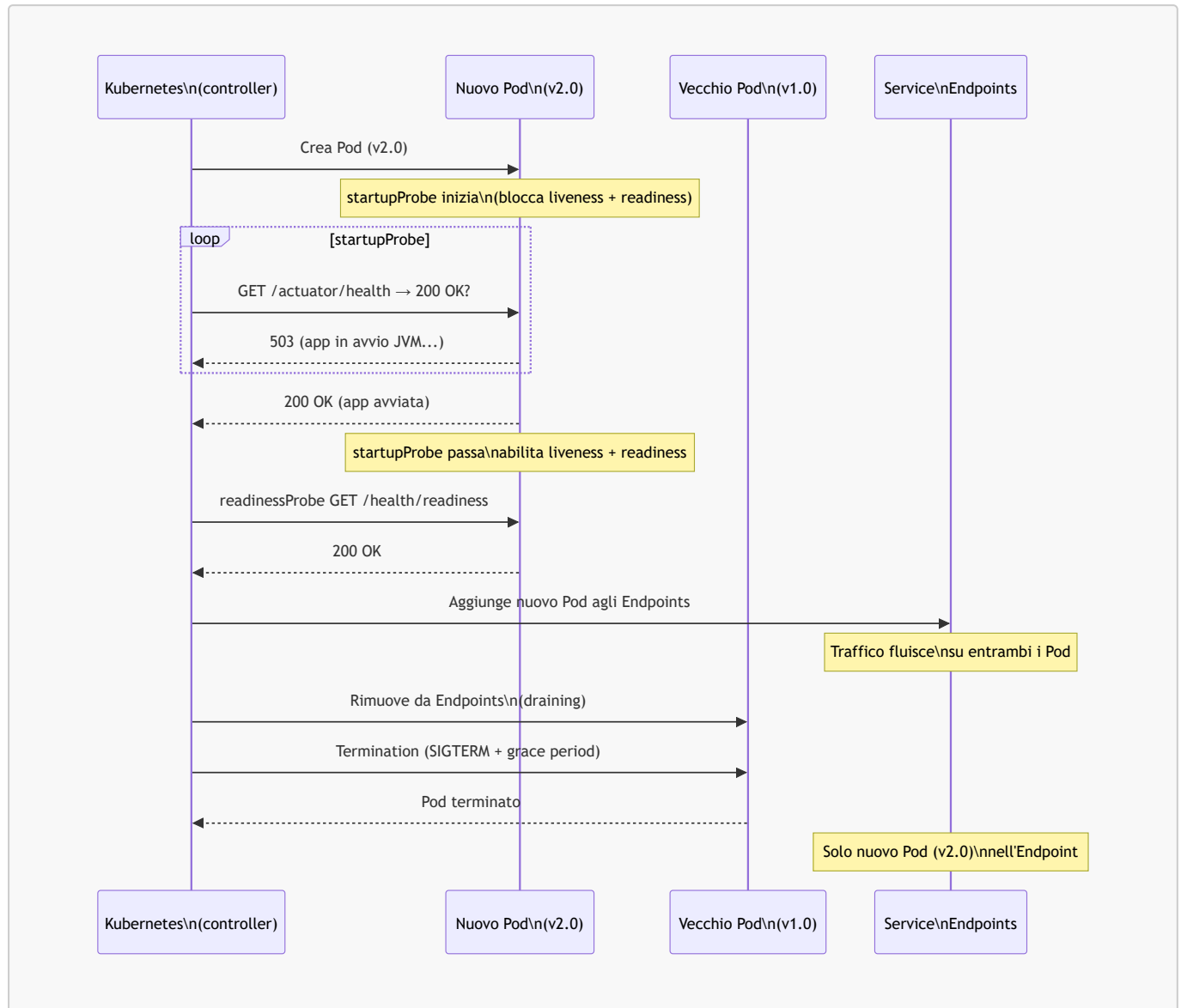
Durante un rolling update, la `readinessProbe` è il segnale che determina il progresso:

1. Kubernetes crea un nuovo Pod (nuova versione)
2. Aspetta che `readinessProbe` passi → Pod diventa Ready
3. Il Pod viene aggiunto agli Endpoint del Service
4. Kubernetes rimuove il vecchio Pod dagli Endpoint
5. (ripete per ogni replica)

Se `readinessProbe` non passa mai, il rolling update si blocca (rimane alla versione precedente). Questo è il comportamento corretto: protegge da deploy di versioni rotte.

Diagrammi

Sequenza di Health Check durante Rolling Update



Comandi Pratici con Output Atteso

```
# Creare un Deployment con tutte e tre le probe
oc apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: spring-app
  namespace: myproject
spec:
  replicas: 2
  selector:
    matchLabels:
      app: spring-app
  template:
    metadata:
      labels:
        app: spring-app
    spec:
      containers:
      - name: app
        image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
        command: ["/bin/sh", "-c", "sleep 30 && while true; do sleep 10; done"]
        resources:
          requests: {memory: "128Mi", cpu: "100m"}
          limits: {memory: "256Mi", cpu: "500m"}
        startupProbe:
          exec:
            command: ["/bin/sh", "-c", "[ -f /tmp/ready ]"]
          failureThreshold: 30
          periodSeconds: 5
        livenessProbe:
          exec:
            command: ["/bin/sh", "-c", "pgrep sleep"]
          periodSeconds: 10
          failureThreshold: 3
        readinessProbe:
          exec:
            command: ["/bin/sh", "-c", "[ -f /tmp/ready ]"]
          periodSeconds: 5
          failureThreshold: 3
EOF

# Output atteso:
# deployment.apps/spring-app created
```

```
# Verificare lo stato delle probe in un Pod
oc describe pod -l app=spring-app -n myproject | grep -A20 "Containers:"
```

```
# Output atteso (estratto):
# Containers:
#   app:
#     Liveness:      exec [/bin/sh -c pgrep sleep] delay=0s timeout=1s period=10s
#success=1 #failure=3
#     Readiness:     exec [/bin/sh -c [ -f /tmp/ready ]] delay=0s timeout=1s
#period=5s #success=1 #failure=3
#     Startup:       exec [/bin/sh -c [ -f /tmp/ready ]] delay=0s timeout=1s
#period=5s #success=1 #failure=30
```

```
# Forzare il Pod in stato Not Ready (simula app non pronta)
oc exec deployment/spring-app -n myproject -- rm -f /tmp/ready 2>/dev/null || true

# Verificare che il Pod sia uscito dagli Endpoints del Service
oc get endpoints -n myproject
# Output atteso:
# NAME           ENDPOINTS           AGE
# spring-app     (nessun IP)         5m ← Pod rimosso dagli Endpoints!
```

```
# Ripristinare il Pod in stato Ready
oc exec deployment/spring-app -n myproject -- touch /tmp/ready

# Verificare dopo ~10s
oc get endpoints -n myproject
# Output atteso:
# NAME           ENDPOINTS           AGE
# spring-app     10.128.2.15:8080,10.128.3.7:8080 5m
```

```
# Verificare gli eventi di riavvio per probe failures
oc get events -n myproject --field-selector reason=Unhealthy --sort-
by='.lastTimestamp'

# Output atteso (esempio):
# LAST SEEN   TYPE      REASON      OBJECT                MESSAGE
# 2m          Warning   Unhealthy   pod/spring-app-...    Liveness probe failed:
...
# 1m          Warning   Unhealthy   pod/spring-app-...    Readiness probe failed:
...
```

```
# Contare i riavvii dei container per probe liveness failures
oc get pods -n myproject -o wide

# Output atteso:
# NAME                                READY   STATUS    RESTARTS   AGE
```

```
# spring-app-7f6d9b4c8-xz9kp    1/1    Running    3        10m    ← 3 riavvii da liveness
```

```
# Modificare i parametri della startupProbe sul Deployment esistente
oc patch deployment spring-app \
  --type=json \
  -
p='[{"op":"replace","path":"/spec/template/spec/containers/0/startupProbe/failureThreshhold","value":60}]' \
  -n myproject

# Output atteso:
# deployment.apps/spring-app patched
```

Esempi YAML Completi

Deployment Spring Boot con Tutte e Tre le Probe

```
# Deployment completo per applicazione Spring Boot su ARO
# Con startupProbe per JVM warm-up, readinessProbe per dipendenze, livenessProbe
anti-deadlock
apiVersion: apps/v1
kind: Deployment
metadata:
  name: spring-boot-app
  namespace: myproject
  labels:
    app: spring-boot-app
    version: "2.1.0"
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0 # ← zero downtime: sempre almeno
replicas Pod ready
      maxSurge: 1 # ← crea 1 Pod extra durante il rolling
update
  selector:
    matchLabels:
      app: spring-boot-app
  template:
    metadata:
      labels:
        app: spring-boot-app
    spec:
      terminationGracePeriodSeconds: 60 # ← 60s per completare richieste in
corso (graceful shutdown)
      containers:
        - name: app
          image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
          command: ["/bin/sh", "-c", "sleep 3600"]
          ports:
            - containerPort: 8080
              name: http
          resources:
            requests:
              memory: "512Mi"
              cpu: "500m"
            limits:
              memory: "1Gi"
              cpu: "2000m"
          env:
            - name: JAVA_TOOL_OPTIONS
              value: "-XX:MaxRAMPercentage=75.0 -XX:+UseContainerSupport"
```

```

- name: SERVER_PORT
  value: "8080"

# --- startupProbe: blocca liveness+readiness durante il bootstrap JVM ---
# Max tempo di avvio: 30 × 10s = 5 minuti
startupProbe:
  httpGet:
    path: /actuator/health          # ← endpoint base, non verifica
dipendenze
    port: 8080
    scheme: HTTP
    failureThreshold: 30            # ← 30 tentativi × 10s = 5 min max
    periodSeconds: 10
    timeoutSeconds: 5              # ← timeout singola call (JVM potrebbe
essere lenta)
    # initialDelaySeconds: NON necessario con startupProbe

# --- livenessProbe: rileva deadlock (NON verifica dipendenze esterne!) --
-
livenessProbe:
  httpGet:
    path: /actuator/health/liveness # ← endpoint SENZA dipendenze esterne
    port: 8080
    scheme: HTTP
    periodSeconds: 10
    timeoutSeconds: 3
    failureThreshold: 3             # ← 3 × 10s = 30s prima del riavvio
    successThreshold: 1            # ← basta 1 successo per tornare
healthy

# --- readinessProbe: verifica prontezza al traffico (può verificare
dipendenze) ---
readinessProbe:
  httpGet:
    path: /actuator/health/readiness # ← endpoint che verifica DB, cache,
ecc.
    port: 8080
    scheme: HTTP
    periodSeconds: 5               # ← check ogni 5s (più frequente di
liveness)
    timeoutSeconds: 3
    failureThreshold: 3            # ← dopo 3 fallimenti → rimosso da
Endpoints
    successThreshold: 1

---
# Service che usa la readinessProbe per dirigere il traffico
apiVersion: v1
kind: Service
metadata:
  name: spring-boot-app
  namespace: myproject
spec:
  selector:
    app: spring-boot-app          # ← solo Pod con readinessProbe passing

```

```
ricevono traffico
ports:
- name: http
  port: 80
  targetPort: 8080
type: ClusterIP
```

Deployment con Probe tcpSocket e exec

```
# Esempi di probe con handler diversi da httpGet
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres-proxy
  namespace: myproject
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres-proxy
  template:
    metadata:
      labels:
        app: postgres-proxy
    spec:
      containers:
      - name: pgbouncer
        image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
        command: ["/bin/sh", "-c", "sleep 3600"]
        resources:
          requests:
            memory: "64Mi"
            cpu: "50m"
          limits:
            memory: "128Mi"
            cpu: "200m"
        ports:
        - containerPort: 5432
          name: postgres

      # tcpSocket: verifica che la porta PostgreSQL sia aperta
      livenessProbe:
        tcpSocket:
          port: 5432
          # ← tenta connessione TCP alla porta
          5432

        initialDelaySeconds: 15
        periodSeconds: 10
        failureThreshold: 3

      # exec: esegue un comando di health check custom
```

```
readinessProbe:
  exec:
    command:
      - "/bin/sh"
      - "-c"
      # Verifica che pgbouncer risponda con psql
      - "psql -U pgbouncer -c 'SHOW POOLS;' 2>/dev/null | grep -q 'pool'"
  periodSeconds: 5
  timeoutSeconds: 3
  failureThreshold: 3
```

Note Specifiche per Azure ARO

startupProbe Obbligatoria per Applicazioni Java su ARO

Su ARO, le applicazioni Spring Boot / Quarkus / WildFly hanno tempi di avvio variabili a seconda del carico del nodo. La **startupProbe** è obbligatoria per evitare riavvii prematuri da parte della **livenessProbe**:

```
# Pattern raccomandato per app Java su ARO (nodi Azure con burst CPU)
startupProbe:
  httpGet:
    path: /q/health/started      # ← endpoint Quarkus startup
    port: 8080
  failureThreshold: 30          # ← 5 minuti massimi
  periodSeconds: 10
  timeoutSeconds: 10           # ← timeout più alto per JVM cold start
```

 **Suggerimento:** Per applicazioni Spring Boot 3.x, abilitare gli endpoint di Actuator separati: `management.endpoint.health.probes.enabled=true`. Questo crea `/actuator/health/liveness` e `/actuator/health/readiness` indipendenti.

Timeout su Azure Load Balancer

Azure Load Balancer ha un timeout di inattività TCP di **4 minuti** (default). Le sonde readiness che usano **tcpSocket** con connessioni keep-alive devono configurare `timeoutSeconds < 240`:

```
# Verificare il timeout del Load Balancer ARO
az network lb probe list \
  --resource-group <aro-infra-rg> \
  --lb-name <aro-internal-lb> \
  --query "[].{Name:name, Protocol:protocol, IntervalInSeconds:intervalInSeconds}" \
  --output table
```

Configurazione Probe per Route HTTPS su ARO

Quando il container serve HTTPS direttamente (TLS termination nel Pod), la `httpGet` probe deve usare `scheme: HTTPS`:

```
livenessProbe:
  httpGet:
    path: /health
    port: 8443
    scheme: HTTPS          # ← necessario se il container serve HTTPS
  periodSeconds: 10
```

Verifica Probe Failures nel Logging ARO

Su ARO, i fallimenti delle probe generano eventi Kubernetes. Per un monitoring proattivo, integrare con Azure Monitor:

```
# Elencare tutti gli eventi Unhealthy delle ultime 2 ore
oc get events -n myproject \
  --field-selector reason=Unhealthy \
  --sort-by='.lastTimestamp' \
  | tail -20

# Verificare il numero di riavvii per probe liveness nel namespace
oc get pods -n myproject \
  -o custom-
columns='NAME:.metadata.name,RESTARTS:.status.containerStatuses[0].restartCount' \
  | sort -t' ' -k2 -rn | head -10
```

Riepilogo dei Punti Chiave

- La **livenessProbe** riavvia il container se fallisce: usarla **solo** per rilevare stati irrecuperabili (deadlock); non includere dipendenze esterne per evitare riavvii a cascata
 - La **readinessProbe** rimuove il Pod dagli Endpoint del Service se fallisce: il Pod rimane in esecuzione e può tornare ready senza riavvio
 - La **startupProbe** è obbligatoria per applicazioni con avvio lento (JVM): blocca liveness e readiness durante il bootstrap, evitando riavvii prematuri
 - Il **tempo massimo di avvio** garantito da startupProbe è $\text{failureThreshold} \times \text{periodSeconds}$ secondi
 - Durante il rolling update, la **readinessProbe** determina quando il nuovo Pod riceve traffico: se non passa mai, il deploy si blocca (protezione da versioni rotte)
 - Per Spring Boot su ARO: usare `/actuator/health/liveness` (senza dipendenze) per liveness e `/actuator/health/readiness` (con dipendenze) per readiness
-

Domande di Verifica

1. Qual è la conseguenza del fallimento della **livenessProbe** per **failureThreshold** volte consecutive?

- A) Il Pod viene rimosso dagli Endpoint del Service ma continua a girare
- B) Il Deployment scala a zero repliche automaticamente
- C) Il kubelet riavvia il container all'interno del Pod ☒
- D) Viene creato un nuovo Pod e il vecchio viene cancellato

2. Un'applicazione Spring Boot impiega mediamente 3 minuti per avviarsi sulla JVM. Quale configurazione di **startupProbe** garantisce fino a 5 minuti di avvio?

- A) `failureThreshold: 5, periodSeconds: 60`
- B) `failureThreshold: 30, periodSeconds: 10` ☒
- C) `failureThreshold: 50, periodSeconds: 6`
- D) `initialDelaySeconds: 300, periodSeconds: 10`

3. Durante un rolling update, un nuovo Pod non supera mai la **readinessProbe**. Cosa accade al rolling update?

- A) Kubernetes forza il completamento del rolling update dopo 10 minuti
- B) Il rolling update si blocca: i vecchi Pod rimangono in esecuzione (nessun downtime) ☒
- C) Kubernetes cancella i vecchi Pod e aspetta che il nuovo Pod si avvii
- D) Kubernetes fa rollback automatico alla versione precedente

4. Quale handler di probe è appropriato per verificare la disponibilità di un server PostgreSQL su porta 5432?

- A) `httpGet` verso `http://postgres:5432`
- B) `exec` con `pg_isready -h postgres -p 5432`
- C) `tcpSocket` sulla porta 5432 ☒
- D) `grpc` con health check standard

5. Qual è l'anti-pattern principale nell'uso della **livenessProbe**?

- A) Usare `httpGet` invece di `exec`
- B) Impostare `periodSeconds` inferiore a 5
- C) Configurare la **livenessProbe** su un endpoint che verifica le dipendenze esterne (es. database) ☒
- D) Usare `failureThreshold: 3` (troppo permissivo)